

MLOps gobernado por especificaciones

Spec-Driven Development aplicado al ciclo de vida de modelos
sobre Databricks, Unity Catalog y Azure DevOps

Documento técnico extendido

Versión ampliada del marco de adopción

Este documento desarrolla en profundidad el marco presentado en la propuesta resumida. Incluye fundamentos, anatomía de cada contrato, gobierno de identidad, patrones de CI/CD y evidencia de implementación.

Contents

1	Introducción	3
1.1	Propósito	3
1.2	Alcance	3
1.3	Cómo leer este documento	3
2	Fundamento: Spec-Driven Development	3
2.1	Definición	3
2.2	Por qué es pertinente en MLOps	3
2.3	Distinción entre lineamiento y especificación	4
3	Arquitectura de referencia	5
3.1	Capas	5
3.2	Separación por ambiente	5
4	Gobierno de identidad	6
4.1	Por qué la identidad es parte de la especificación	6
4.2	Tipos de identidad y cuándo usar cada una	6
4.3	Federación de identidad para CI/CD	6
4.4	ETL y procesos de datos bajo service principal	6
5	Estructura del proyecto	7
5.1	Organización del repositorio	7
5.2	Convenciones de nombres	8
6	Los contratos en detalle	8
6.1	Contrato de datos	8
6.2	Contrato de características	8
6.3	Contrato de entrenamiento	9
6.4	Contrato de validación	9
6.5	Contrato de despliegue	10
6.6	Contrato de monitoreo	10
7	Integración y entrega continuas	10
7.1	Los tres pipelines	10
7.2	Elementos de configuración	11
7.3	Validación de precondiciones del ambiente	11
8	Promoción de modelos	11
8.1	Patrones posibles	11

8.2	El patrón adoptado	11
8.3	Alias y ciclo campeón/retador	12
9	Inferencia, observabilidad y consumo	12
9.1	Inferencia por lotes y en tiempo real	12
9.2	La tabla de inferencia como evidencia	12
9.3	Monitoreo	13
9.4	Consumo asistido por agentes	14
10	Roles y responsabilidades	15
11	Ruta de adopción	15
12	Riesgos y controles	16
13	Supuestos y límites	16
14	Glosario	17
A	Anexo: implementación de referencia, paso a paso	18
A.1	Separación por ambiente	18
A.2	Materialización de recursos desde la especificación	21
A.3	Contrato de características	22
A.4	Entrenamiento, trazabilidad y veredicto	22
A.5	Registro y promoción gobernada	24
A.6	Observabilidad: dependencias y condiciones	24
A.7	Servicio en tiempo real	27
A.8	Consumo asistido por agentes	28
A.9	Respuestas y secuencia de razonamiento	29
A.10	Observaciones de la implementación	31

1. Introducción

1.1 Propósito

Este documento describe un marco de trabajo para operar el ciclo de vida completo de modelos de *machine learning* bajo el principio de que **las especificaciones y los contratos gobiernan el proceso**, y no la disciplina individual de quien lo ejecuta.

El punto de partida es una situación frecuente en organizaciones con práctica analítica madura pero sin industrialización: un equipo de Data Science que entrena, valida, registra y promueve modelos de forma manual. El problema no es la calidad del trabajo, sino su forma. Cuando el criterio técnico reside en personas y notebooks, el resultado no es reproducible, no es auditable, no admite delegación y no escala.

1.2 Alcance

El marco cubre desde la definición del producto de datos hasta el consumo de las predicciones, incluyendo:

- Contratos de datos, características, entrenamiento, validación, despliegue y monitoreo.
- Empaquetado declarativo de recursos y promoción entre ambientes.
- Gobierno de identidad: *service principals*, identidades administradas y federación.
- Integración y entrega continuas sobre Azure DevOps.
- Observabilidad de modelos y de la capa conversacional que los consume.

Queda fuera de alcance el diseño matemático de los modelos, la arquitectura de la plataforma de datos fuera del ámbito analítico, y las herramientas de MLOps ajenas al ecosistema descrito.

1.3 Cómo leer este documento

La sección 2 establece el fundamento conceptual. Las secciones 3 a 8 desarrollan el marco por capas. Las secciones 9 a 11 tratan operación, roles y adopción. El anexo reúne la evidencia de una implementación de referencia.

2. Fundamento: Spec-Driven Development

2.1 Definición

Spec-Driven Development (SDD) es una práctica en la que la especificación es el artefacto primario del que se derivan la implementación, las pruebas y las decisiones de despliegue. Un cambio de comportamiento comienza por un cambio en la especificación; el código es consecuencia, no origen.

La idea tiene raíces en el enfoque *contract-first* de las APIs, donde primero se define el contrato y después se implementa, y en *Design by Contract*, donde las precondiciones y poscondiciones son parte de la definición del componente. Se relaciona con TDD y BDD, aunque con una diferencia relevante: en aquellas prácticas la especificación son las pruebas; en SDD la especificación es un documento estructurado de más alto nivel, del cual las pruebas son una derivación entre varias.

2.2 Por qué es pertinente en MLOps

En el desarrollo de software convencional, el código determina el comportamiento. En *machine learning* el comportamiento depende además de los datos, y esa dependencia introduce tres

propiedades que hacen a SDD particularmente adecuado:

1. **El artefacto no es determinista respecto al código.** Dos ejecuciones del mismo código sobre datos distintos producen modelos distintos. Sin una especificación de qué condiciones debe cumplir el resultado, no hay forma de decidir automáticamente si ese modelo puede desplegarse.
2. **La degradación es silenciosa.** Un modelo que empeora no lanza excepciones: sigue prediciendo. Solo una comparación explícita contra umbrales declarados convierte esa degradación en una señal accionable.
3. **El criterio de aceptación es de negocio, no de ingeniería.** Qué recall mínimo es aceptable, qué segmentos no pueden degradarse y cuánto puede costar un falso positivo son decisiones del dueño del producto. Declararlas en un archivo versionado las hace revisables por quien corresponde.

2.3 Distinción entre lineamiento y especificación

Muchas organizaciones ya disponen de lineamientos de MLOps: documentos que describen qué debe cumplirse en cada fase. Son valiosos y necesarios, pero tienen un límite estructural: **describen la norma sin poder hacerla cumplir.**

La diferencia operativa se concentra en dos puntos. El primero es **cuándo se detecta el incumplimiento**: un lineamiento se verifica por revisión, de modo que una desviación pasa inadvertida salvo que alguien la busque; una especificación se verifica en cada ejecución, y una desviación detiene el proceso. El segundo es **qué constituye la evidencia**: en un caso es un acta o una lista de verificación firmada, que documenta que alguien afirmó haber revisado; en el otro es el registro de la ejecución, que documenta qué se comprobó y con qué resultado.

La propuesta de este marco es precisamente ese tránsito: conservar íntegro el criterio ya definido en los lineamientos y dotarlo de un mecanismo de cumplimiento.

3. Arquitectura de referencia

3.1 Capas

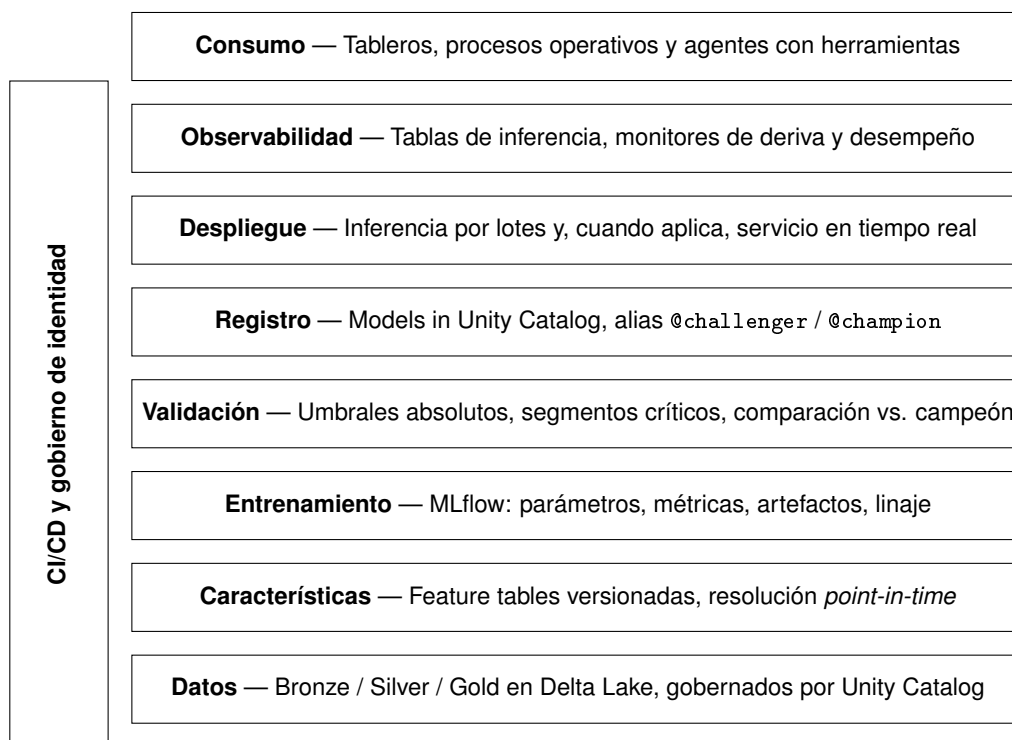


Figure 1: Capas del marco. CI/CD y gobierno de identidad son transversales: ninguna capa se modifica manualmente en producción.

3.2 Separación por ambiente

Se definen tres ambientes —desarrollo, pruebas y producción— y cada uno corresponde a un **catálogo distinto de Unity Catalog**. La elección del catálogo como frontera, en lugar del esquema o del espacio de trabajo, responde a que el catálogo es la unidad natural de permisos, auditoría y linaje.

Ambiente	Catálogo	Quién escribe	Propósito
Desarrollo	gold_adv_dev	Científico de datos	Experimentación, diseño de features, iteración sobre hipótesis
Pruebas	gold_adv_qa	Service principal de CI/CD	Ejecución integrada del ciclo, validación técnica y de negocio
Producción	gold_adv_prd	Service principal de producción	Inferencia, monitoreo y consumo. No entrena

Table 1: Ambientes y responsabilidad de escritura. El científico de datos conserva lectura sobre producción para diagnóstico.

El acceso de solo lectura del equipo de ciencia de datos al catálogo de producción es deseable: permite comparar un modelo candidato contra el vigente en igualdad de condiciones. Cuando restricciones de seguridad lo impiden, el patrón alternativo es generar instantáneas controladas de datos productivos hacia desarrollo, preservando representatividad sin romper los controles de acceso.

4. Gobierno de identidad

4.1 Por qué la identidad es parte de la especificación

La identidad bajo la cual se ejecuta un proceso determina qué puede leer, qué puede escribir y qué queda registrado en la auditoría. Tratarla como configuración de infraestructura, ajena al producto, produce dos patologías conocidas: procesos productivos atados a la cuenta personal de quien los creó, y permisos concedidos de forma incremental y no documentada hasta exceder ampliamente lo necesario.

Por eso la identidad y los roles requeridos se declaran en la especificación del producto, junto al resto de los contratos. El equipo de plataforma sigue siendo quien los crea y otorga —la segregación de funciones lo exige— pero la solicitud deja de ser un ticket informal y pasa a ser un artefacto revisable en el *pull request*.

4.2 Tipos de identidad y cuándo usar cada una

Identidad	Uso	Consideración
Usuario	Solo desarrollo interactivo	Nunca en QA ni producción: el proceso se rompe al cambiar la persona de equipo
Service principal	Ejecución de jobs en QA y producción; conexión desde CI/CD	Requiere gestión del secreto o, preferiblemente, federación sin secreto
Identidad administrada asignada por el usuario	Recursos de cómputo que acceden a almacenamiento u otros servicios	Recurso independiente: sobrevive a la destrucción del cómputo al que se adjunta
Identidad administrada asignada por el sistema	Poco recomendable en pipelines efímeros	Se destruye con el recurso, junto con sus asignaciones de rol

Table 2: Selección de identidad según el contexto de ejecución.

4.3 Federación de identidad para CI/CD

Para la conexión entre la plataforma de CI/CD y la nube se recomienda **federación de identidad de carga de trabajo** (OIDC) en lugar de un secreto de cliente. El servicio de CI/CD presenta un token de corta vida emitido por su propio proveedor de identidad, que la nube valida contra una relación de confianza previamente establecida. El resultado es que no existe ninguna credencial de larga duración almacenada en variables del pipeline.

Cuando la federación no está disponible, el secreto debe residir en un almacén gestionado y referenciarse por nombre, nunca incrustarse en código, notebooks ni archivos del repositorio.

4.4 ETL y procesos de datos bajo service principal

El mismo criterio aplica aguas arriba. Los procesos de ingesta y transformación que alimentan las capas Silver y Gold deben ejecutarse bajo service principal, por tres razones:

1. **Continuidad operativa.** Un proceso de ingesta atado a una persona es un punto único de fallo organizativo.
2. **Linaje coherente.** Si el ETL escribe con una identidad y el entrenamiento lee con otra sin relación declarada, la trazabilidad se fragmenta. Con identidades de servicio nombradas por

dominio, el linaje de Unity Catalog reconstruye la cadena completa desde la fuente hasta la predicción.

3. **Permisos mínimos verificables.** Un principal por dominio con permisos explícitos sobre sus esquemas permite auditar el acceso real, en lugar de heredarlo del perfil amplio de un usuario.

En la práctica esto significa un principal por dominio y ambiente —por ejemplo, ingesta de operaciones en producción— con permisos de lectura sobre las fuentes y escritura acotada a los esquemas que le corresponden. El contrato de datos del producto de ML declara qué tablas consume y, por tanto, qué permisos necesita el principal que lo ejecuta.

The screenshot shows the 'Jobs & Pipelines' interface in Databricks. It includes a 'Create new' section with options for 'Ingestion pipeline', 'ETL pipeline', and 'Job'. Below this is a filter bar with options like 'All', 'Jobs', 'Pipelines', and 'Accessible by me'. The main table lists jobs with columns for Name, Type, Tags, Run as, Trigger, and Recent runs. The 'Recent runs' column shows a sequence of status icons (green for success, red for failure, grey for paused). The last job, 'flight_delay_model_training_job_qa', is shown with a green icon and a tooltip that says 'Activar Windows. Ve a Configuración para activar Windows.'

Name	Type	Tags	Run as	Trigger	Recent runs
flight_delay_model_batch_scoring_job...	Job		fernandogomezroj...	Paused - Schedul...	— — — — —
flight_delay_model_batch_scoring_job...	Job		fernandogomezroj...	Paused - Schedul...	— — — — —
flight_delay_model_batch_scoring_job...	Job		fernandogomezroj...	Paused - Schedul...	— — — — —
flight_delay_model_feature_materializa...	Job		fernandogomezroj...	Paused - Schedul...	— — — — —
flight_delay_model_feature_materializa...	Job		fernandogomezroj...	Paused - Schedul...	— — — — —
flight_delay_model_feature_materializa...	Job		fernandogomezroj...	Paused - Schedul...	— — — — —
flight_delay_model_model_promotion...	Job		fernandogomezroj...	Paused - Schedul...	— — — — —
flight_delay_model_model_promotion...	Job		fernandogomezroj...	Paused - Schedul...	— — — — —
flight_delay_model_training_job_dev	Job		fernandogomezroj...	Paused - Schedul...	— — — — —
flight_delay_model_training_job_prod	Job		fernandogomezroj...	Paused - Schedul...	— — — — —
flight_delay_model_training_job_qa	Job		fernandogomezroj...	Paused - Schedul...	— — — — —

Figure 2: Jobs materializados desde la especificación en los tres ambientes. La columna de identidad de ejecución hace visible bajo qué principal corre cada proceso. El job de entrenamiento del ambiente productivo figura declarado pero sin ejecuciones: producción no entrena.

5. Estructura del proyecto

5.1 Organización del repositorio

Cada producto de ML es autocontenido y desplegable de forma independiente. La estructura separa lógica pura —testeable sin plataforma— de los puntos de entrada que se ejecutan como tareas.

```
producto_ml/
databricks.yml # especificacion de despliegue (targets, computo, identidad)
openspec/config.yaml # especificacion del producto (dominio, area, catalogos)
resources/ # jobs declarados: training, inferencia, monitor, promocion
azure-pipelines/ # ci, cd-training, cd-scoring
src/producto_ml/
  data_prep/ # extraccion, validacion, particion temporal
  features/ # definitions.py (contrato), store.py, validation.py
  training/ # train.py (logica pura), hpo.py
  evaluation/ # thresholds.py (contrato), metrics.py, verdict.py
  deployment/ # promocion, inferencia por lotes
  agent/ # herramientas y orquestacion de consumo asistido
tests/unit/ # pruebas sobre logica pura, sin dependencia de plataforma
```

Listing 1: Estructura de referencia de un producto de ML.

La separación entre lógica pura y notebooks no es estética. Permite que la integración continua ejecute pruebas unitarias en segundos sobre las funciones de cálculo de *features*, métricas y veredicto, sin levantar cómputo ni acceder a datos.

5.2 Convenciones de nombres

Las convenciones se declaran una vez en la especificación del producto y se derivan mecánicamente al resto de artefactos. Esto elimina la deriva de nombres entre equipos y hace predecible la ubicación de cualquier recurso.

Elemento	Patrón	Ejemplo
Catálogo por ambiente	gold_adv_{ambiente}	gold_adv_qa
Esquema	{area}_{proyecto}	operaciones_retrasos
Modelo en Unity Catalog	md_{area}_{sujeto}	md_operaciones_retraso
Experimento MLflow	/ml/{dominio}/{area}/{proyecto}	ml/ops/operaciones/retrasos
Tabla de features	{catalogo}.{esquema}.features	—
Tabla de inferencia	{modelo}_inference	md_operaciones_retraso_inference

Table 3: Convenciones derivadas de la especificación del producto.

6. Los contratos en detalle

6.1 Contrato de datos

Declara qué se consume y bajo qué condiciones se considera apto. Su función es detener la ejecución antes de entrenar sobre datos que no cumplen, en lugar de producir un modelo cuya degradación se descubrirá semanas después.

```
fuelle: silver.operaciones.segmentos
llave: id_segmento
columna_fecha: fecha_operacion
frescura_max: 24h
columnas:
  - {nombre: hora_salida, tipo: int, rango: [0, 23], nulos: 0%}
  - {nombre: distancia_km, tipo: double, rango: [0, 20000], nulos: <1%}
  - {nombre: retraso_previo, tipo: double, rango: [-60, 600], nulos: <5%}
disponibilidad_en_inferencia:
  # Campos conocidos al momento de predecir. Los ausentes aquí NO pueden
  # usarse como features: su uso constituye fuga temporal.
  - hora_salida
  - distancia_km
  - retraso_previo
```

Listing 2: Elementos mínimos de un contrato de datos.

La sección de disponibilidad en inferencia merece énfasis. La causa más frecuente de modelos con métricas excelentes que fracasan en producción es el uso de información que no existe en el momento de predecir. Declararlo explícitamente convierte un error sutil en una validación automática.

6.2 Contrato de características

Define el catálogo de variables, su origen y su tipo, y —de forma crítica— establece que **la misma función de cálculo se usa en entrenamiento y en inferencia**. Duplicar esa lógica en dos lugares garantiza que eventualmente divergirán; el resultado es un modelo que recibe en producción variables calculadas de manera distinta a como aprendió.

El almacén de características aporta además resolución *point-in-time*: al construir el conjunto de entrenamiento, cada observación se une con el valor de la característica vigente en su momento, no con el valor actual. Sin este mecanismo, la fuga temporal es prácticamente inevitable en cualquier dominio con evolución histórica.

```
CATALOGO_FEATURES = [
    Feature("hora_salida", origen="hora_salida", tipo="int64",
            descripcion="Hora programada de salida"),
    Feature("retraso_previo", origen="retraso_previo", tipo="float64",
            descripcion="Retraso del segmento anterior de la aeronave"),
    Feature("congestion", origen="congestion", tipo="int64",
            descripcion="Operaciones programadas en la misma franja y origen"),
]

def calcular_features(df):
    """Contrato: misma logica en entrenamiento e inferencia."""
    faltantes = [c for c in COLUMNAS_REQUERIDAS if c not in df.columns]
    if faltantes:
        raise ValueError(f"Columnas fuente faltantes: {faltantes}")
    ...
```

Listing 3: Contrato de características como estructura declarativa.

6.3 Contrato de entrenamiento

Declara qué hace reproducible el proceso: tabla de características, columnas utilizadas, variable objetivo, estrategia de partición, semillas y métrica principal. Un modelo debe poder reconstruirse a partir de su código, sus datos de entrada y su configuración registrada.

La partición merece mención aparte. En problemas con componente temporal, la partición aleatoria produce métricas optimistas sin valor predictivo: el modelo aprende de datos futuros para predecir el pasado. La especificación debe fijar partición temporal y, cuando el volumen lo permite, validación cruzada con ventanas móviles.

6.4 Contrato de validación

Es el núcleo del gobierno. Declara bajo qué condiciones un modelo puede ser candidato a producción, y produce un veredicto binario que el pipeline respeta sin intervención humana.

```
UMBRALES = {
    "auc": 0.75, # metrica principal
    "recall": 0.60, # guardrail: costo alto del falso negativo
    "precision": 0.50,
}

# Ningun segmento critico del negocio puede degradarse por debajo de su minimo,
# aunque la metrica global cumpla.
UMBRALES_POR_SEGMENTS = {
    "franja_pico": {"auc": 0.70, "recall": 0.55},
    "franja_valle": {"auc": 0.70, "recall": 0.55},
    "operacion_fin_de_semana": {"auc": 0.70, "recall": 0.55},
}

# El candidato puede ser marginalmente inferior al campeon sin ser rechazado,
# para evitar bloqueos por ruido estadistico.
TOLERANCIA_VS_CAMPEON = 0.02
```

Listing 4: Contrato de validación con umbrales, segmentos y tolerancia.

Tres decisiones de diseño explican esta estructura. Los **umbrales absolutos** evitan que un modelo malo se promueva solo por ser mejor que otro peor. Los **segmentos críticos** impiden que un promedio aceptable oculte la degradación en una ruta, un mercado o una cohorte relevante. La **tolerancia frente al campeón** reconoce que exigir mejora estricta en cada iteración bloquea el ciclo por variación aleatoria.

6.5 Contrato de despliegue

El proyecto se empaqueta como un paquete declarativo de recursos. Un único archivo describe jobs, experimentos, modelos registrados y monitores; la herramienta de línea de comandos los materializa. Ningún recurso productivo se crea manualmente.

```
targets:
  qa:
    variables:
      catalogo: gold_adv_qa
      cluster: <id-cluster-qa>
    run_as:
      service_principal_name: sp-<dominio>-qa

  prod:
    mode: production
    variables:
      catalogo: gold_adv_prd
      cluster: <id-cluster-prd>
    run_as:
      service_principal_name: sp-<dominio>-prd
    permissions:
      - {group_name: grp-<dominio>-owner, level: CAN_MANAGE}
      - {group_name: grp-<dominio>-consumer, level: CAN_VIEW}
    # El job de entrenamiento NO se incluye en este target.
    # Produccion recibe el modelo por promocion, nunca por entrenamiento local.
```

Listing 5: Anatomía de la especificación de despliegue por ambiente.

La consecuencia práctica es que el cómputo, el catálogo y la identidad son parámetros del ambiente. Migrar de cómputo provisionado a elástico, o de una topología de catálogos a otra, no requiere tocar la lógica ni los contratos.

6.6 Contrato de monitoreo

Declara qué se vigila y con qué cadencia sobre la tabla de inferencia. Los segmentos monitoreados deben coincidir con los declarados como críticos en la validación: si el criterio de aceptación y el de vigilancia divergen, el modelo puede degradarse justamente donde más importa sin que ninguna alerta se dispare.

7. Integración y entrega continuas

7.1 Los tres pipelines

Pipeline	Disparador	Contenido
ci	Pull request	Estilo y formato; pruebas unitarias sobre lógica pura; validación sintáctica y semántica de la especificación de despliegue; análisis estático de seguridad con puerta de calidad. Bloquea el <i>merge</i> .
cd-training	Merge principal	a Despliegue del paquete en QA; ejecución del ciclo completo: materialización de características, entrenamiento, validación contra contrato y registro con alias en el catálogo de QA.
cd-scoring	Rama versión	de Despliegue en producción; aprobación manual en un ambiente protegido; promoción del modelo validado; habilitación de inferencia y activación del monitor.

Table 4: Pipelines, disparadores y responsabilidades.

7.2 Elementos de configuración

- **Grupos de variables** por ambiente, que resuelven el punto final de la plataforma y la referencia a la credencial. Nunca contienen secretos en texto plano.
- **Ambientes protegidos** con aprobadores designados. La aprobación de producción es un control de negocio, no técnico, y debe recaer en quien responde por el resultado.
- **Conexión de servicio** con federación de identidad, evitando secretos de larga duración.
- **Puerta de calidad** que impide el *merge* ante vulnerabilidades críticas o cobertura insuficiente.

7.3 Validación de precondiciones del ambiente

Un riesgo recurrente en la adopción es que configuraciones temporales de prueba alcancen producción: umbrales relajados para depurar el ciclo, datos sintéticos, o identidades deshabilitadas para facilitar una ejecución local. Estas configuraciones suelen quedar documentadas como comentarios en el código, es decir, como **promesas humanas y no como contratos ejecutables**.

El marco incorpora una verificación de precondiciones en el despliegue de producción, que rechaza el destino si detecta alguna de las siguientes condiciones:

- Uso de datos sintéticos habilitado en el ambiente productivo.
- Umbrales por debajo del mínimo declarado en la especificación del producto.
- Identidad de ejecución ausente o correspondiente a un usuario.
- Monitor no declarado para el modelo que se despliega.
- Permisos de grupo no definidos.

8. Promoción de modelos

8.1 Patrones posibles

Patrón	Qué se promueve	Consideración
Implementación de código	El código de entrenamiento; cada ambiente entrena con sus datos	Recomendado: facilita el reentrenamiento y somete todo el ciclo a los mismos controles
Implementación de modelo	El artefacto ya entrenado en desarrollo	Útil si el entrenamiento es muy costoso; dificulta el reentrenamiento automático y el gobierno del código auxiliar
Híbrido	Código hasta QA; artefacto hacia producción	Reduce costo de cómputo en producción a cambio de mayor complejidad operativa

Table 5: Patrones de promoción y sus compromisos.

8.2 El patrón adoptado

Se adopta la implementación de código con una restricción adicional: **el entrenamiento queda excluido del ambiente de producción**. Producción no entrena; recibe mediante una operación de copia la versión que obtuvo veredicto favorable en QA, preservando el identificador de ejecución y las etiquetas de trazabilidad del original.

Esta decisión tiene dos efectos. El primero es que el artefacto en producción es **idéntico**, no equivalente, al que superó la validación: no existe la posibilidad de que un reentrenamiento en

producción produzca un modelo distinto del aprobado. El segundo es que se elimina la ruta alternativa: si el job de entrenamiento no existe en el destino productivo, nadie puede registrar un modelo allí saltándose la aprobación.

8.3 Alias y ciclo campeón/retador

Los alias del registro sustituyen a los estados fijos tradicionales y permiten expresar el ciclo con precisión:

- @challenger — versión que superó la validación y aspira a reemplazar a la vigente.
- @champion — versión que atiende las decisiones de negocio en cada ambiente.

Cuando el caso lo justifica, el retador puede operar en sombra —recibiendo tráfico real sin que sus predicciones tengan efecto— para comparar su comportamiento contra el campeón bajo condiciones de producción antes de la sustitución.

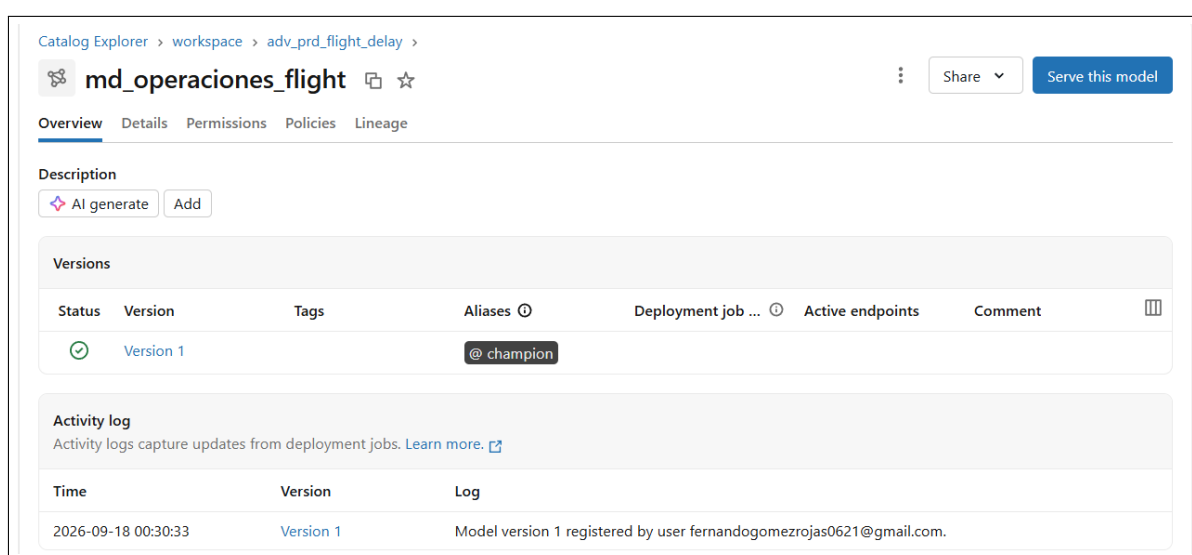


Figure 3: Modelo en el catálogo de producción con alias @champion. El registro de actividad documenta el origen de la versión: no fue entrenada allí, sino copiada desde el ambiente de pruebas tras obtener veredicto favorable.

9. Inferencia, observabilidad y consumo

9.1 Inferencia por lotes y en tiempo real

La elección no es de preferencia técnica sino de naturaleza de la decisión. Si la decisión se consume por lotes o a través de tableros, la inferencia programada es adecuada y notablemente más económica. Si la decisión ocurre en el momento de un evento y requiere latencia de milisegundos, se justifica un servicio en línea.

El servicio en tiempo real introduce una exigencia adicional: las características deben resolverse en el momento de la solicitud, lo que requiere un almacén de características en línea sincronizado con el almacén histórico. Adoptarlo sin esa pieza produce un endpoint que solo acepta variables ya calculadas, trasladando el problema al consumidor.

9.2 La tabla de inferencia como evidencia

Cada ejecución de inferencia escribe la predicción junto con la marca temporal y **el identificador de la versión del modelo que la produjo**. Esta última columna es la que convierte un registro

operativo en evidencia de auditoría: permite reconstruir, para cualquier decisión pasada, qué versión la generó, con qué datos y bajo qué contrato fue validada.

Columna	Función
Características	Entrada efectiva del modelo; base del perfil estadístico y la deriva
Predicción	Salida del modelo
Marca temporal	Define las ventanas de comparación del monitor
Versión del modelo	Vincula la decisión con el artefacto y su veredicto de validación
Resultado real	Cuando está disponible, habilita el desempeño observado

Table 6: Estructura mínima de la tabla de inferencia.

9.3 Monitoreo

Sobre la tabla de inferencia se declara un monitor que produce tres familias de señales: perfil estadístico de las variables de entrada, deriva entre ventanas temporales, y desempeño observado cuando el resultado real está disponible.

Dos consideraciones prácticas. La **granularidad debe fijarse según el volumen por ventana**: una ventana con pocas observaciones produce métricas de varianza alta que generan alarmas espurias. Y el **rezago de la etiqueta** es la norma en muchos dominios: cuando el resultado real llega semanas después, la deriva de las variables de entrada funciona como señal anticipada del deterioro.

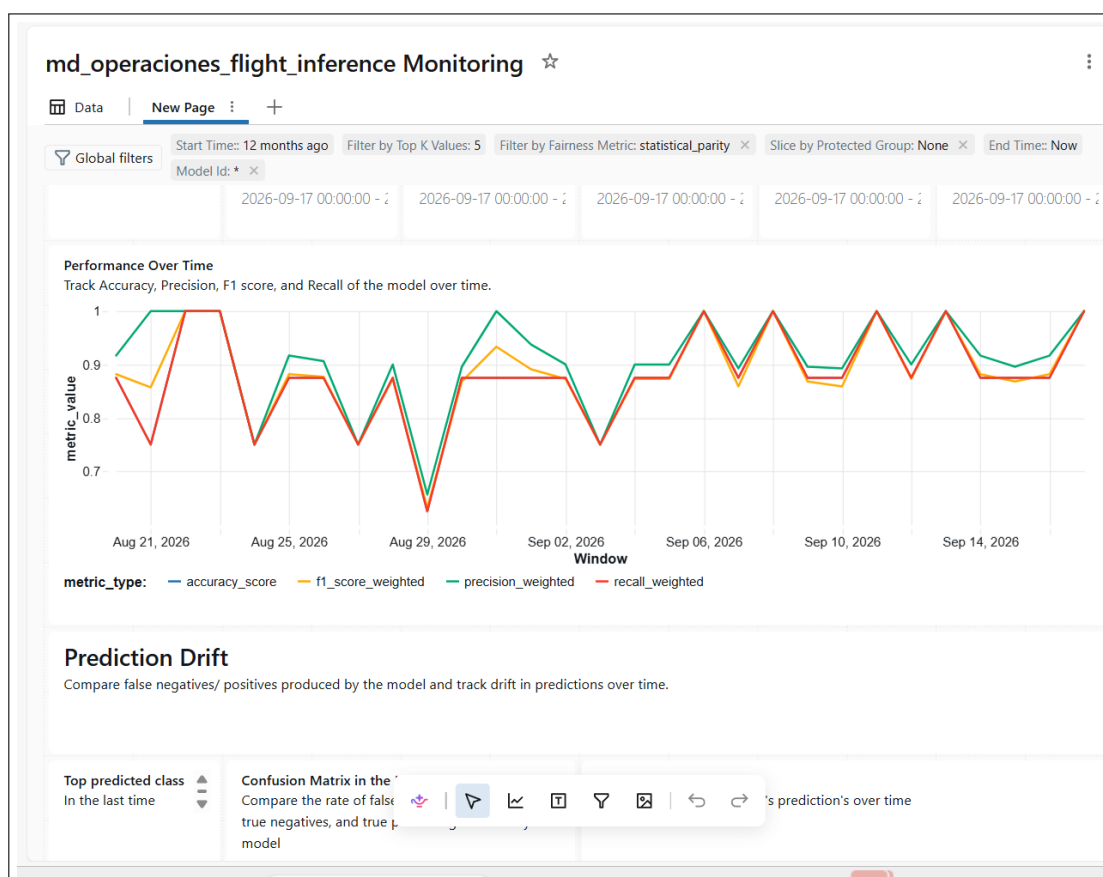


Figure 4: Tablero del monitor declarado en la especificación. Registra desempeño por ventana temporal y deriva de predicción. La volatilidad entre ventanas ilustra el efecto del tamaño de muestra sobre la estabilidad de la métrica: la granularidad debe fijarse según el volumen por período.

9.4 Consumo asistido por agentes

Las predicciones adquieren valor cuando el negocio puede interrogarlas. Un agente conversacional expone el modelo vigente y su tabla de inferencia mediante **herramientas declaradas**: funciones acotadas que ejecutan consultas verificables sobre el catálogo.

El agente opera bajo el mismo principio de gobierno que el resto del ciclo:

- **Solo lectura.** No entrena, no promueve, no escribe.
- **Sin cifras no fundamentadas.** Toda cantidad reportada procede de una herramienta; si no existe herramienta para la pregunta, el agente lo declara en lugar de estimar.
- **Trazabilidad obligatoria.** Cada interacción se registra con las herramientas invocadas, sus argumentos, la latencia y el consumo, de modo que cualquier respuesta sea reconstruible.

La conclusión de diseño es que el gobierno no termina en el modelo. La capa que traduce predicciones a lenguaje natural es también un componente que puede fallar, y requiere contrato y observabilidad con el mismo rigor.



Figure 5: Traza de una consulta al agente. Se registran la pregunta, la herramienta invocada sobre el catálogo y la respuesta devuelta. Cada cifra reportada es reconstruible a partir de la traza.

10. Roles y responsabilidades

Rol		Responsable de	Límite
Científico de datos		Contratos de datos, características, entrenamiento y umbrales técnicos; experimentación en desarrollo	No promueve ni opera producción; acceso de solo lectura al ambiente productivo
Ingeniero de ML		Especificación de despliegue, pipelines, operación de QA y producción, monitoreo y reentrenamiento	No define umbrales de negocio ni aprueba su propia promoción
Plataforma de seguridad	y	Creación de identidades de servicio, asignación de roles, políticas de cómputo y red	No modifica la lógica ni los contratos del producto
Dueño de negocio	de	Criterios de aceptación, definición de segmentos críticos, aprobación de promoción	No interviene en la implementación
Auditoría		Verificación de evidencia y trazabilidad	Consume registros; no ejecuta ni modifica

Table 7: Separación de funciones. Ningún rol concentra definición, validación y promoción.

11. Ruta de adopción

Fase	Objetivo		Actividades	Resultado verificable
1	Versionar y parametrizar	y	Reescribir el entrenamiento manual como código sin valores incrustados; registrar en MLflow	Un modelo reproducible desde el repositorio
2	Declarar contratos		Extraer esquema, características, umbrales y segmentos a archivos versionados	El pipeline falla ante contrato incumplido
3	Empaquetar y automatizar	y	Declarar recursos como paquete; construir la integración continua	Despliegue reproducible en desarrollo y QA
4	Gobernar la promoción	la	Excluir entrenamiento de producción; promover por copia tras aprobación; ejecutar con identidad de servicio	Ninguna ruta a producción sin aprobación
5	Cerrar el ciclo		Monitoreo sobre tablas de inferencia; disparadores de reentrenamiento; consumo asistido	Deterioro detectado antes del reporte del negocio

Table 8: Adopción incremental. Cada fase entrega valor sin requerir la siguiente.

La secuencia importa. Intentar automatizar antes de haber declarado los contratos produce un pipeline que ejecuta rápidas decisiones que nadie revisó; declarar contratos sin versionar el código produce documentación que se desactualiza. El orden propuesto asegura que cada fase se apoye en una base verificable.

12. Riesgos y controles

Riesgo	Efecto	Control
Configuración temporal que alcanza producción	Modelo sin validación real operando sobre decisiones de negocio	Verificación de precondiciones en el despliegue productivo
Fuga de parametrización en librerías compartidas	Rompe la portabilidad sin generar error visible	Despliegue de prueba en ambiente alterno como parte de la integración continua
Fuga temporal en el conjunto de entrenamiento	Métricas optimistas; degradación inmediata en producción	Declaración de disponibilidad en inferencia; partición temporal obligatoria
Divergencia entre cálculo de características en entrenamiento e inferencia	El modelo recibe variables distintas de las que aprendió	Función única compartida; resolución <i>point-in-time</i>
Etiqueta real con rezago	El desempeño observado no puede calcularse en la ventana corriente	Deriva de entrada como señal anticipada; recálculo al llegar la etiqueta
Granularidad de monitoreo inadecuada al volumen	Alarmas espurias que erosionan la confianza en el monitor	Selección de ventana según observaciones por período
Dependencia del equipo de identidad	Bloquea el despliegue en QA y producción	Identidades declaradas en la especificación y solicitadas por revisión con antelación
Concentración de funciones	Quien define el umbral aprueba su propio cumplimiento	Separación de roles con aprobación de negocio en la promoción

Table 9: Riesgos priorizados y mecanismos de control.

13. Supuestos y límites

- El marco asume un catálogo de datos unificado con control de acceso y linaje, separación por catálogo entre ambientes, y una plataforma de CI/CD con soporte para ambientes protegidos y aprobaciones.
- La creación de identidades de servicio y la asignación de roles corresponden al equipo de plataforma. El marco lo asume como restricción de diseño y declara la identidad requerida como parte de la especificación.
- El marco gobierna el ciclo de vida; no sustituye el criterio de modelado. Define dónde se declaran los umbrales y cómo se hacen cumplir, no cuáles deben ser: esos provienen del negocio.
- El patrón descrito asume inferencia por lotes como caso predominante. El servicio en tiempo real está contemplado y se habilita por parámetro del ambiente, pero exige resolución de características en línea y un régimen distinto de latencia y costo.
- La automatización no elimina la aprobación humana en la promoción a producción. La transforma de un acto de confianza en una decisión informada por evidencia.
- El marco reduce la probabilidad de error sistemático, no la elimina. Su valor reside en que los errores que ocurran sean detectables, atribuibles y corregibles.

14. Glosario

Término	Definición
Spec-Driven Development Contrato	Práctica en que la especificación es el artefacto primario del que derivan implementación, pruebas y decisiones de despliegue Especificación verificable por máquina que condiciona la ejecución de una etapa del ciclo
Paquete declarativo	Definición versionada de los recursos de un producto, materializable por herramienta
Alias de registro	Etiqueta móvil sobre una versión de modelo que expresa su papel operativo
Deriva	Cambio en la distribución de las variables de entrada o de las predicciones entre ventanas temporales
Resolución <i>point-in-time</i>	Unión de observaciones con el valor de característica vigente en su momento, evitando fuga temporal
Ejecución en sombra	Operación de un modelo candidato sobre tráfico real sin efecto sobre las decisiones
Veredicto	Resultado binario de la evaluación de un modelo contra su contrato de validación

A. Anexo: implementación de referencia, paso a paso

Este anexo documenta una implementación completa del marco sobre un caso de predicción de retraso operativo. Sigue el orden real del ciclo: provisionamiento, separación por ambiente, materialización de características, entrenamiento y validación, registro y promoción, inferencia, observabilidad y consumo asistido.

El propósito no es documentar un caso de negocio, sino mostrar qué artefactos produce el marco cuando opera y cómo cada uno constituye evidencia verificable.

A.1 Separación por ambiente

La separación entre desarrollo, pruebas y producción se materializa antes que cualquier otro artefacto, porque define la frontera de permisos sobre la que descansa el resto del gobierno.

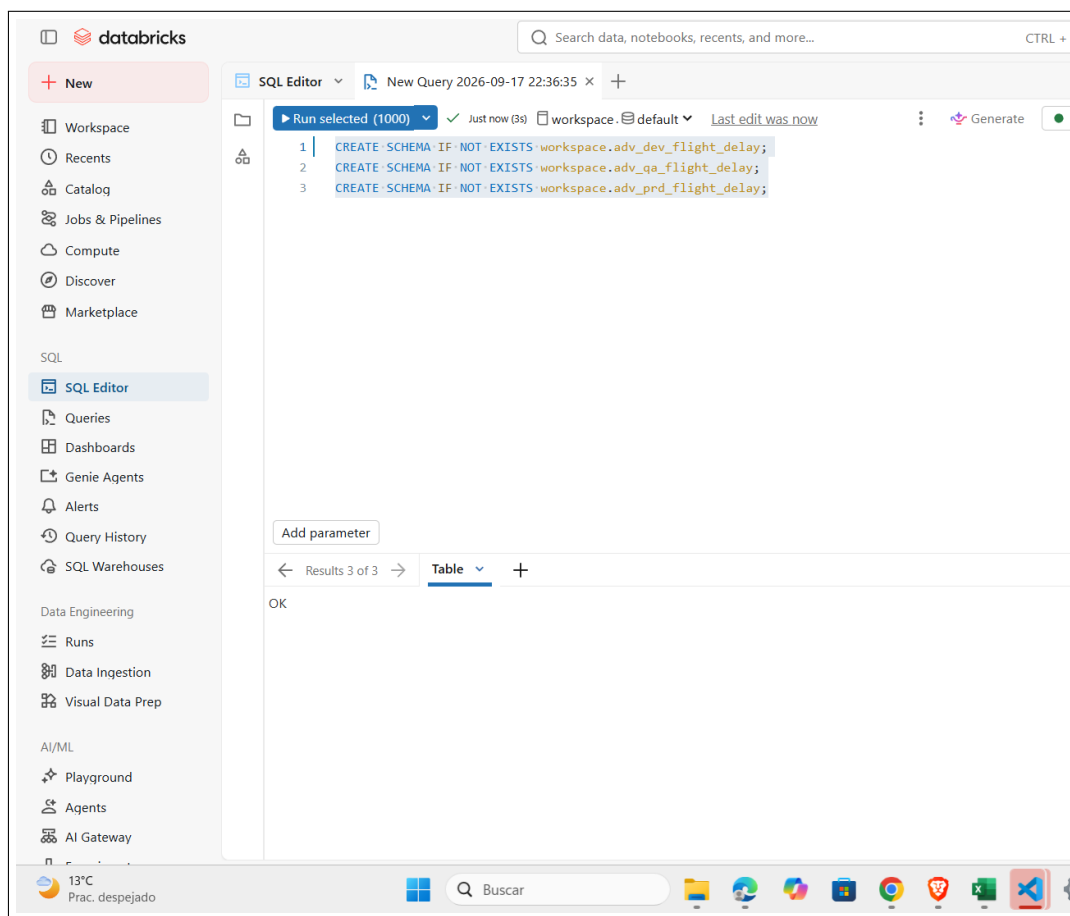


Figure 6: Declaración de los espacios por ambiente. La topología de referencia usa un catálogo por ambiente; cuando las restricciones de la plataforma lo impiden, la separación puede materializarse por esquema sin alterar la especificación del producto: solo cambian dos variables del destino.

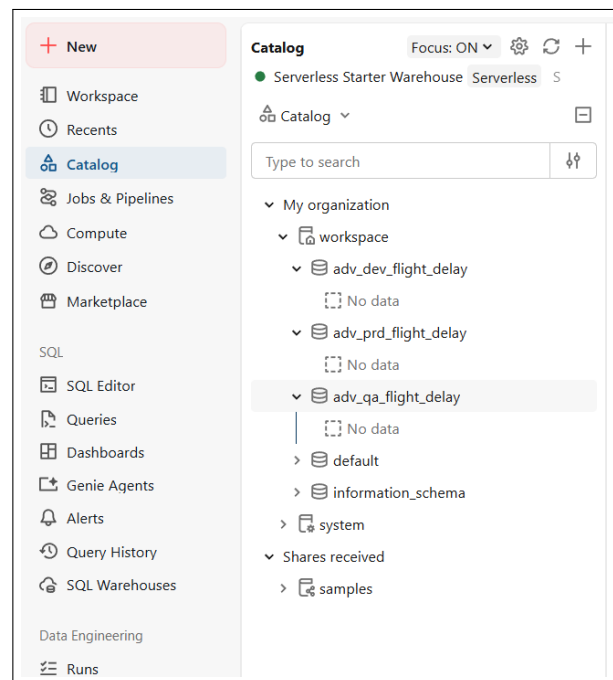


Figure 7: Los tres ambientes visibles en el explorador de datos, inicialmente vacíos. Todo lo que aparezca en ellos será producto de una ejecución declarada, nunca de una intervención manual.

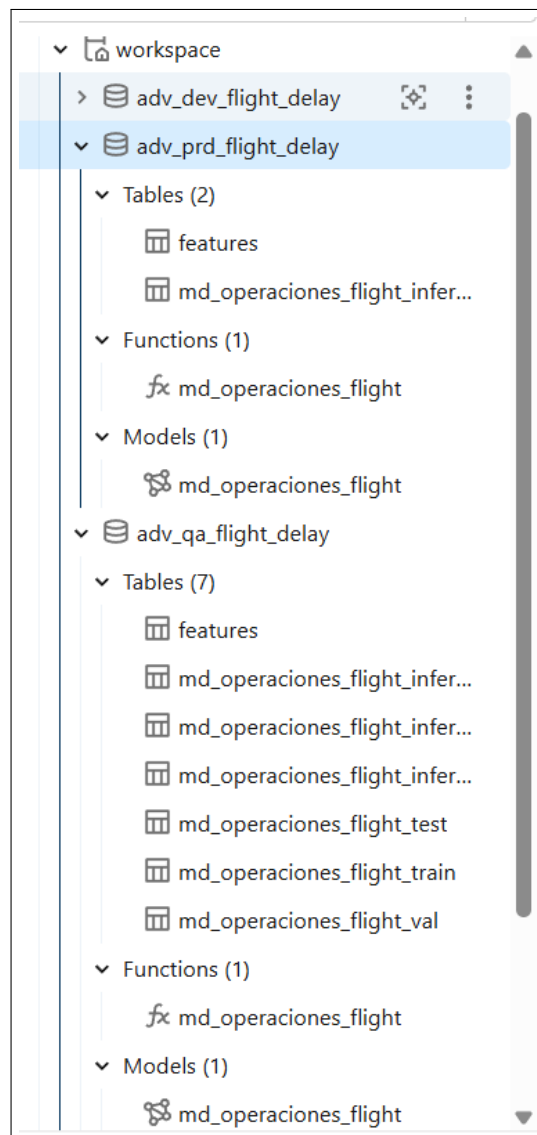


Figure 8: Estado final tras el ciclo completo. El contenido de cada ambiente refleja su función: producción contiene únicamente características e inferencia; los conjuntos de entrenamiento y validación existen solo donde se entrena.

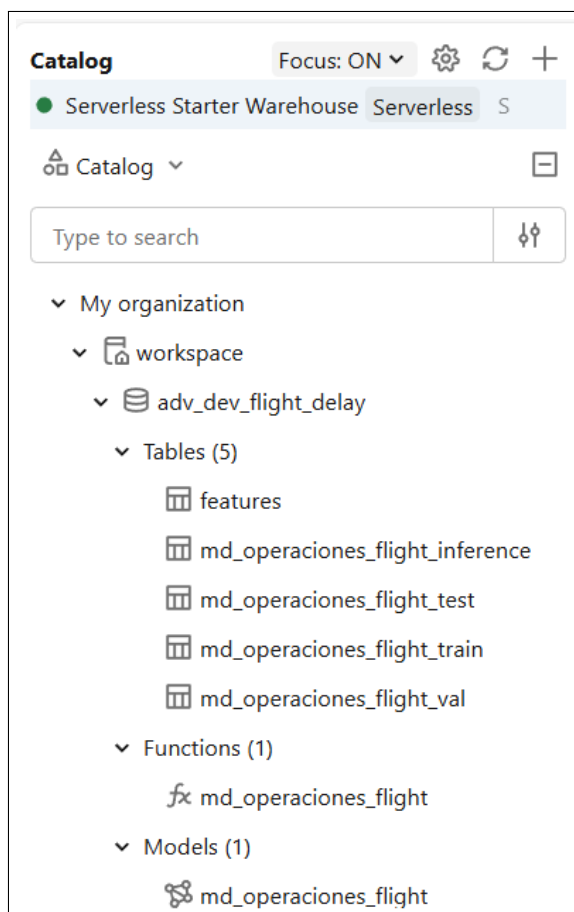


Figure 9: Detalle del ambiente de desarrollo. Coexisten la tabla de características, las particiones de entrenamiento, validación y prueba, la tabla de inferencia y el modelo registrado. La trazabilidad entre ellos es reconstruible por el linaje del catálogo.

A.2 Materialización de recursos desde la especificación

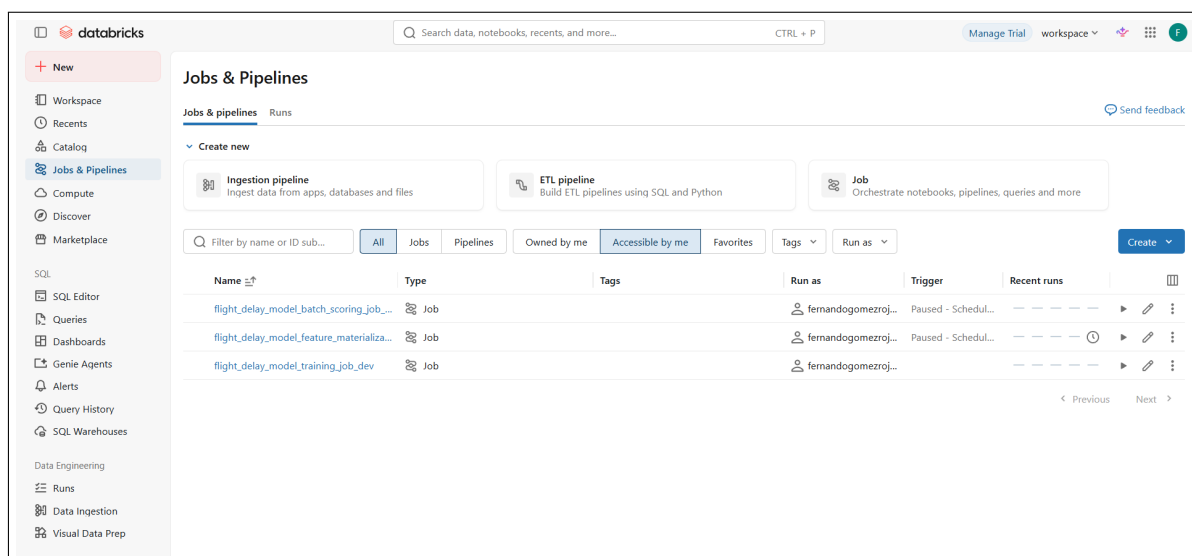
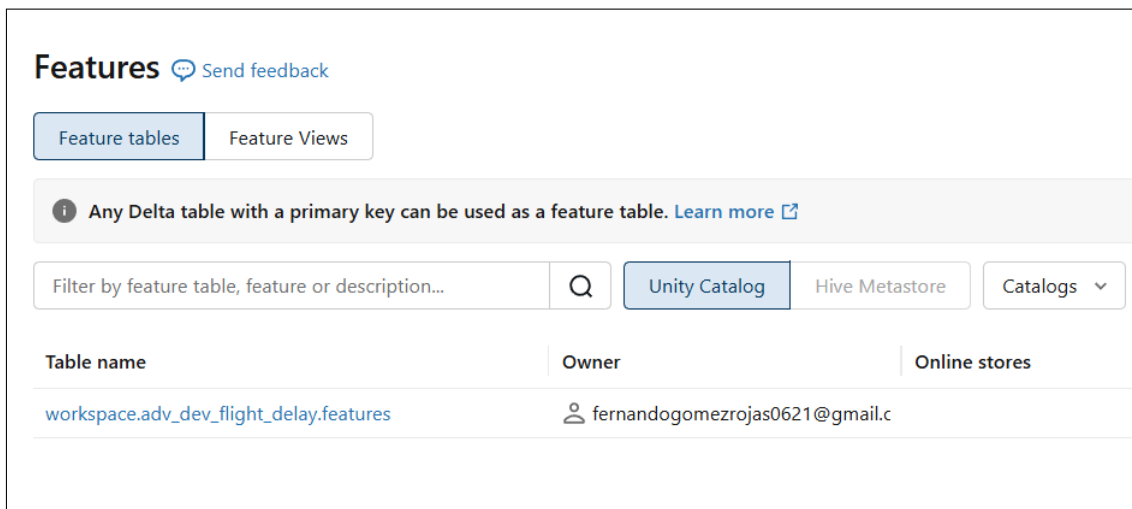


Figure 10: Jobs creados a partir de la especificación de despliegue. Ninguno fue definido en la interfaz: todos derivan del archivo versionado, lo que hace el ambiente reconstruible desde el repositorio.

El estado tras desplegar los tres ambientes se muestra en la figura 2: la columna de identidad de ejecución hace visible bajo qué principal corre cada proceso, y el job de entrenamiento del ambiente productivo figura declarado pero sin ejecuciones.

A.3 Contrato de características



Features Send feedback		
Feature tables Feature Views		
Any Delta table with a primary key can be used as a feature table. Learn more		
Filter by feature table, feature or description... Q Unity Catalog Hive Metastore <input data-cs="2" data-kind="parent" type="button" value="Catalogs"/> v		
Table name	Owner	Online stores
workspace.adv_dev_flight_delay.features	fernandogomezrojas0621@gmail.c	

Figure 11: Tabla de características registrada en el catálogo unificado, con propietario declarado. El registro formal habilita linaje, control de acceso y reutilización entre modelos que compartan variables.

SQL Editor

New Query 2026-09-17 22:36:35

New Query 2026-09-17 23:25:48

Run selected (1000)

Just now (9s)

workspace.adv_dev_flight_delay

Last edit was now

Generate

Serverless Starter ...

Schedule

Save*

Comments

Share

SELECT * FROM workspace.adv_dev_flight_delay.features LIMIT 10;

SELECT COUNT(*), AVG(target) FROM workspace.adv_dev_flight_delay.features;

Add parameter

Table

1

2

3

4

5

6

dep_hour

day_of_week

month

carrier

distance_km

prev_delay

congestion

target

client_id

fecha

1

2

3

4

5

6

18

15

10

10

20

4

5

7

7

11

2

12

3

1

1

1

3

1

435.83980485258985

1941.395757838753

690.3142172917586

778.8414985987877

2602.6746895147985

1693.1240879668865

27.2592114649889...

-6.492221030354489

-9.075184451748235

17.19411910029975

27.7710648493063...

8.805130040009256

14

35

27

20

12

6

0

0

0

1

1

0

0

1

2

3

4

5

2024-01-01

2024-01-01

2024-01-01

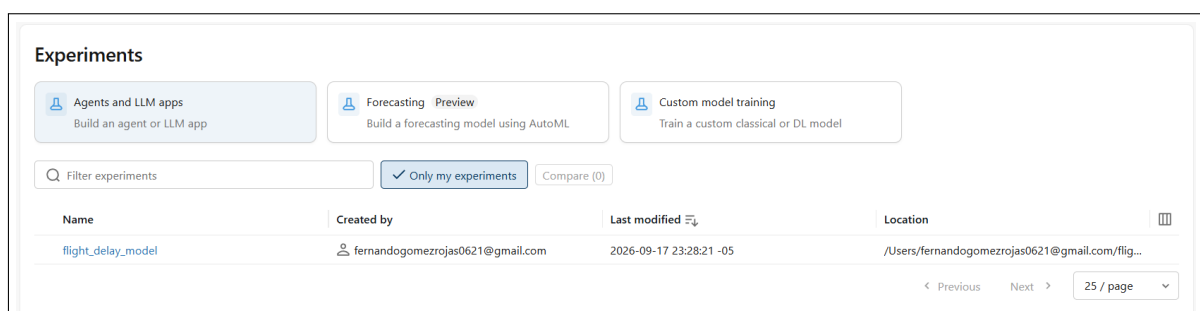
2024-01-01

2024-01-01

2024-01-01

Figure 12: Contenido de la tabla. Las columnas corresponden exactamente a las declaradas en el contrato de características, más la llave de entidad, la columna temporal y la variable objetivo. Es la fuente única que consumen entrenamiento e inferencia.

A.4 Entrenamiento, trazabilidad y veredicto



Experiments			
Agents and LLM apps Build an agent or LLM app			
Forecasting Preview Build a forecasting model using AutoML			
Custom model training Train a custom classical or DL model			
Filter experiments <input checked="" type="button" value="Only my experiments"/> Compare (0)			
Name	Created by	Last modified	Location
flight_delay_model	fernandogomezrojas0621@gmail.com	2026-09-17 23:28:21 -05	/Users/fernandogomezrojas0621@gmail.com/flig...
< Previous Next > 25 / page			

Figure 13: Experimentos generados por el ciclo. Su ruta se deriva de la especificación del producto, de modo que la ubicación de cualquier experimento es predecible sin convención informal.

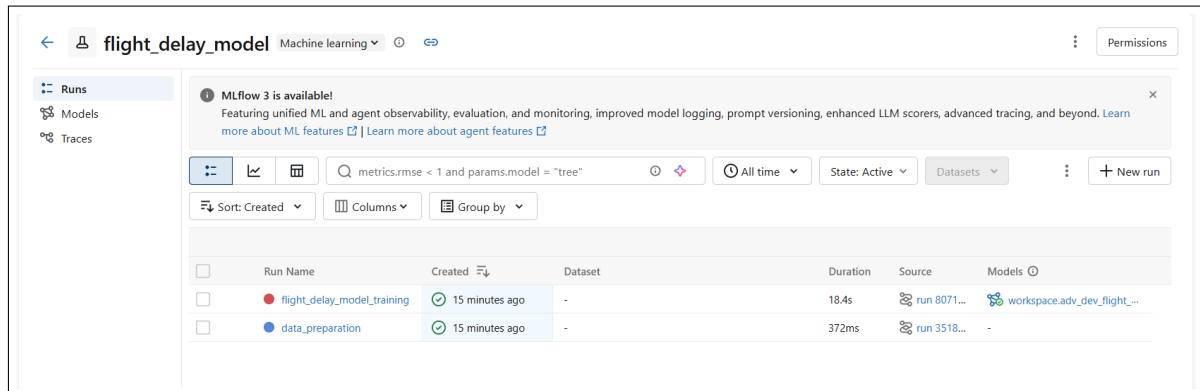


Figure 14: Ejecuciones dentro del experimento. Cada tarea del ciclo registra su propia ejecución, lo que permite aislar dónde se produjo una desviación sin reconstruir el proceso completo.

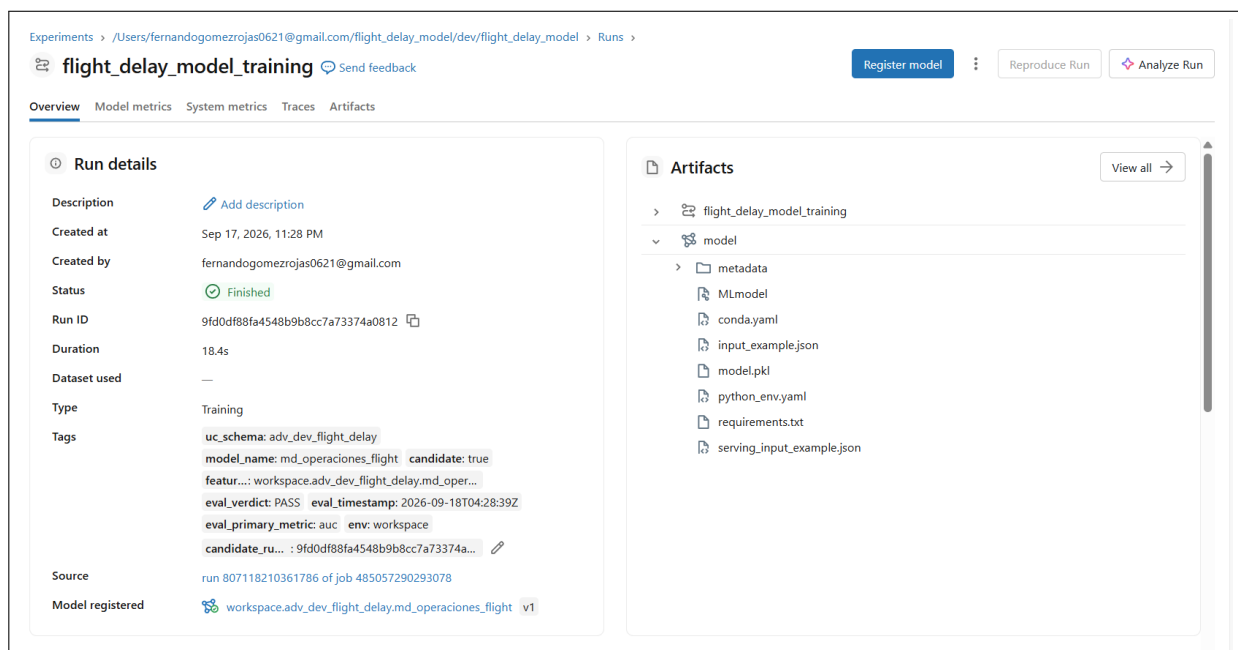


Figure 15: Detalle de la ejecución de entrenamiento. Además de parámetros y artefactos, registra etiquetas de gobierno: el veredicto de validación, la métrica principal empleada y el identificador del candidato. Estas etiquetas son las que vinculan el modelo con el contrato bajo el que fue evaluado, y son verificadas nuevamente antes de la promoción.

A.5 Registro y promoción gobernada

The screenshot shows the 'md_operaciones_flight' model page in the Catalog Explorer. The breadcrumb path is 'Catalog Explorer > workspace > adv_dev_flight_delay >'. The model name is 'md_operaciones_flight' with a share icon and a star. There are tabs for 'Overview', 'Details', 'Permissions', 'Policies', and 'Lineage'. The 'Overview' tab is selected. The description is 'flight_delay_model — ops/operaciones'. Below the description is a 'Versions' table with one row: 'Version 1' with a status icon, a tag '@ champion', and a deployment job link. Below the versions is an 'Activity log' section with a table showing a single activity: 'Model version 1 registered by user fernandogomezrojas0621@gmail.com.' at '2026-09-17 23:28:53'. At the bottom, there is an 'About this model' section with the owner 'fernandogomezrojas0621@gmail.com' and a 'Deployment job' section with a 'Connect deployment job' button.

Catalog Explorer > workspace > adv_dev_flight_delay >

md_operaciones_flight Share Serve this model

Overview Details Permissions Policies Lineage

Description ✎
flight_delay_model — ops/operaciones

Versions

Status	Version	Tags	Aliases	Deployment job ...	Active endpoints	Comment
✓	Version 1		@ champion			

Activity log
Activity logs capture updates from deployment jobs. [Learn more.](#)

Time	Version	Log
2026-09-17 23:28:53	Version 1	Model version 1 registered by user fernandogomezrojas0621@gmail.com.

About this model
Owner fernandogomezrojas0621@gmail.com ✎

Deployment job
To manage the model lifecycle, link a deployment job. A deployment job can automatically trigger steps such as evaluation, approval, and deployment each time a new model version is created. [Learn more.](#)

⇒ Connect deployment job

Figure 16: Modelo registrado en el ambiente de pruebas con alias asignado tras superar la validación. Este registro es el único origen admisible para la promoción a producción.

El resultado de la promoción se muestra en la figura 3: el registro de actividad documenta que la versión no fue entrenada en producción, sino copiada desde pruebas preservando identificador de ejecución y etiquetas.

A.6 Observabilidad: dependencias y condiciones

La instrumentación del monitoreo tiene dos condiciones que conviene conocer antes de adoptarlo, porque determinan el orden de las operaciones.

The screenshot shows the 'md_operaciones_flight_inference' model page in the Catalog Explorer. The breadcrumb path is 'Catalog Explorer > workspace > adv_dev_flight_delay >'. The model name is 'md_operaciones_flight_inference' with a share icon and a star. There are tabs for 'Overview', 'Sample Data', 'Details', 'Permissions', 'Policies', 'History', 'Lineage', 'Insights', and 'Quality'. The 'Quality' tab is selected. The main content area shows a line graph icon and the text 'Enable Data Quality Monitoring for workspace.adv_dev_flight_delay.md_operaciones_flight_inference'. Below this is a button labeled 'Enable'.

Catalog Explorer > workspace > adv_dev_flight_delay >

md_operaciones_flight_inference Open in a dashboard Share Create

Overview Sample Data Details Permissions Policies History Lineage Insights Quality

Enable Data Quality Monitoring for
workspace.adv_dev_flight_delay.md_operaciones_flight_inference
Start monitoring the data quality of this table and receive actionable insights on any anomalies. [Learn more](#)

Enable

Figure 17: Tabla de inferencia sin monitor. La declaración del monitor exige que la tabla exista previamente, de modo que el primer despliegue de un ambiente nuevo debe realizarse, ejecutar una primera inferencia y volver a desplegarse. Es una dependencia real del ciclo, no un defecto de configuración.

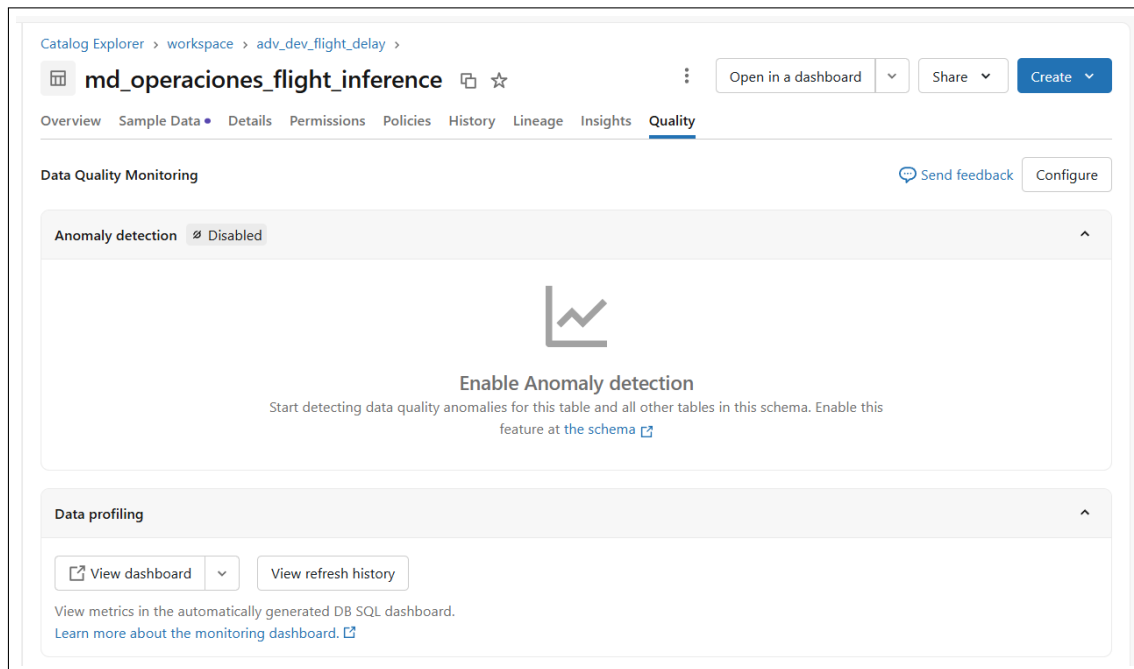


Figure 18: Monitor declarado y activo sobre la tabla. El perfilado queda habilitado con su tablero asociado, generado automáticamente a partir de la especificación.

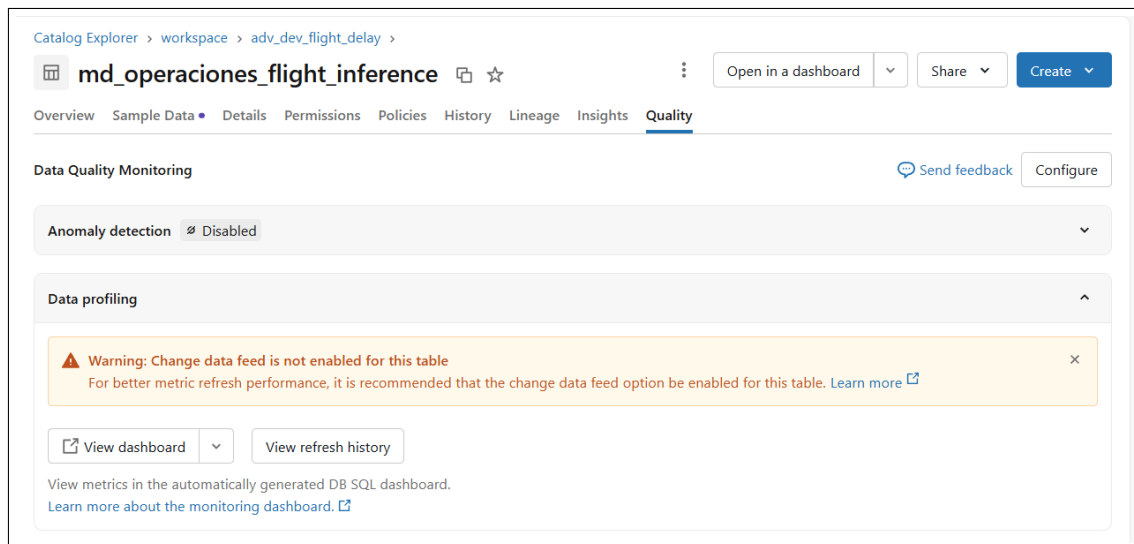


Figure 19: Advertencia sobre la captura de cambios en la tabla. Habilitarla permite que el monitor procese únicamente los registros nuevos en cada actualización, en lugar de recalculando sobre el histórico completo. Es una decisión de costo que escala con el volumen.

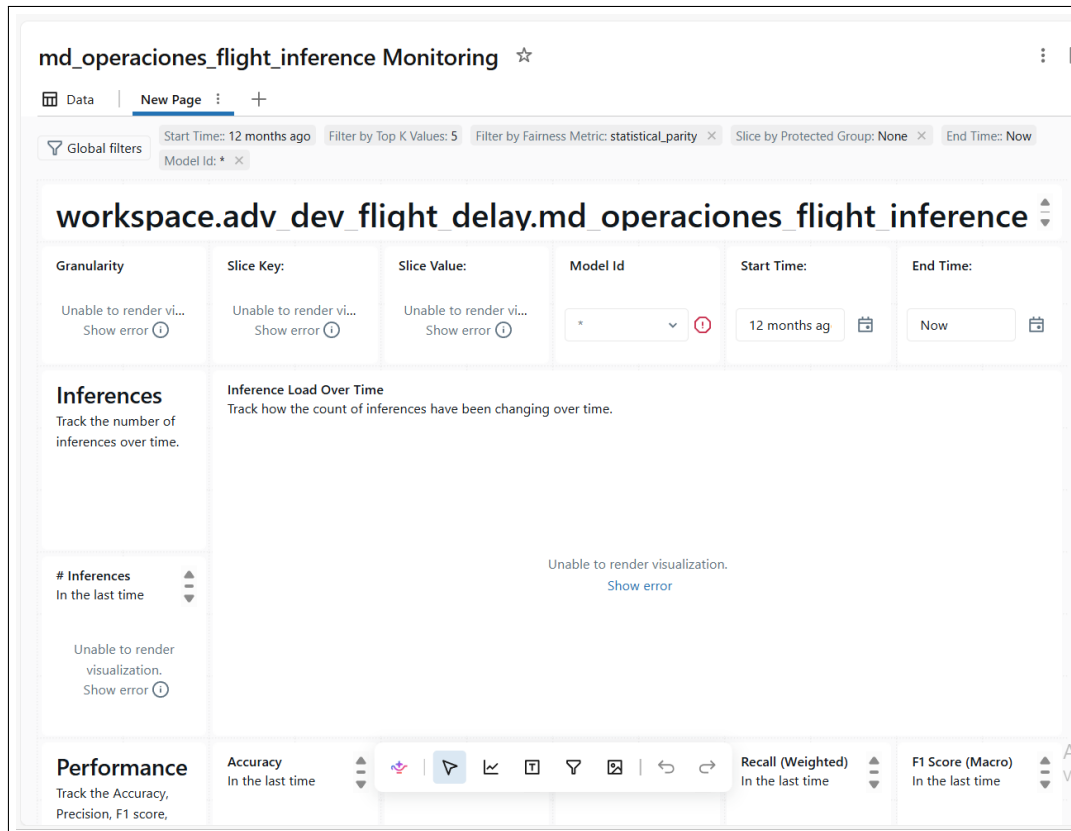


Figure 20: Tablero sin series tras la primera ejecución. Con una única inferencia no existen ventanas temporales comparables, de modo que no puede calcularse deriva. La observabilidad requiere historia, no solo instrumentación: es una propiedad del dato, no de la herramienta.

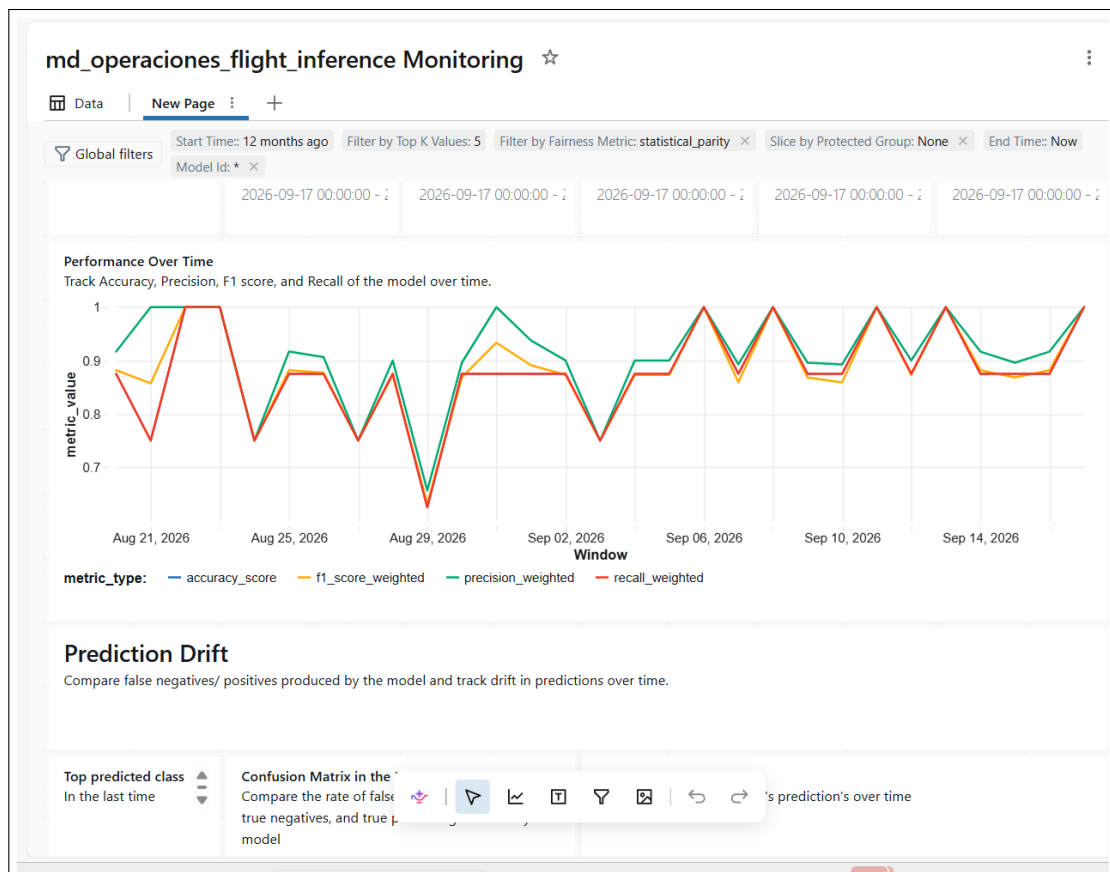


Figure 21: El mismo tablero con historia suficiente. Las series de desempeño y deriva aparecen cuando existen ventanas comparables. La volatilidad entre puntos ilustra el efecto del tamaño de muestra sobre la estabilidad de la métrica: la granularidad debe fijarse según el volumen por período.

La comparación entre las dos figuras anteriores es el punto: el monitor estaba correctamente declarado en ambos momentos. Lo que cambió fue la disponibilidad de historia.

A.7 Servicio en tiempo real

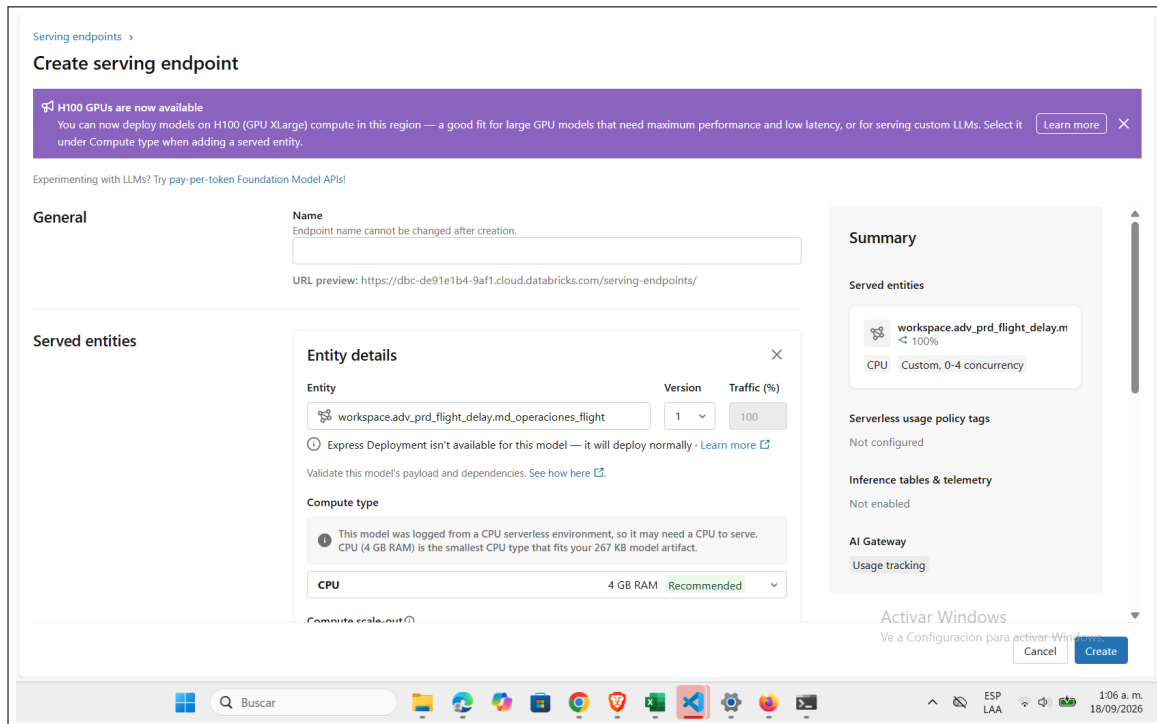


Figure 22: Configuración del servicio en línea sobre el modelo vigente. La habilitación de tablas de inferencia y telemetría es el equivalente en tiempo real de la tabla de inferencia por lotes: sin ella, el servicio sería el único componente del ciclo sin trazabilidad.

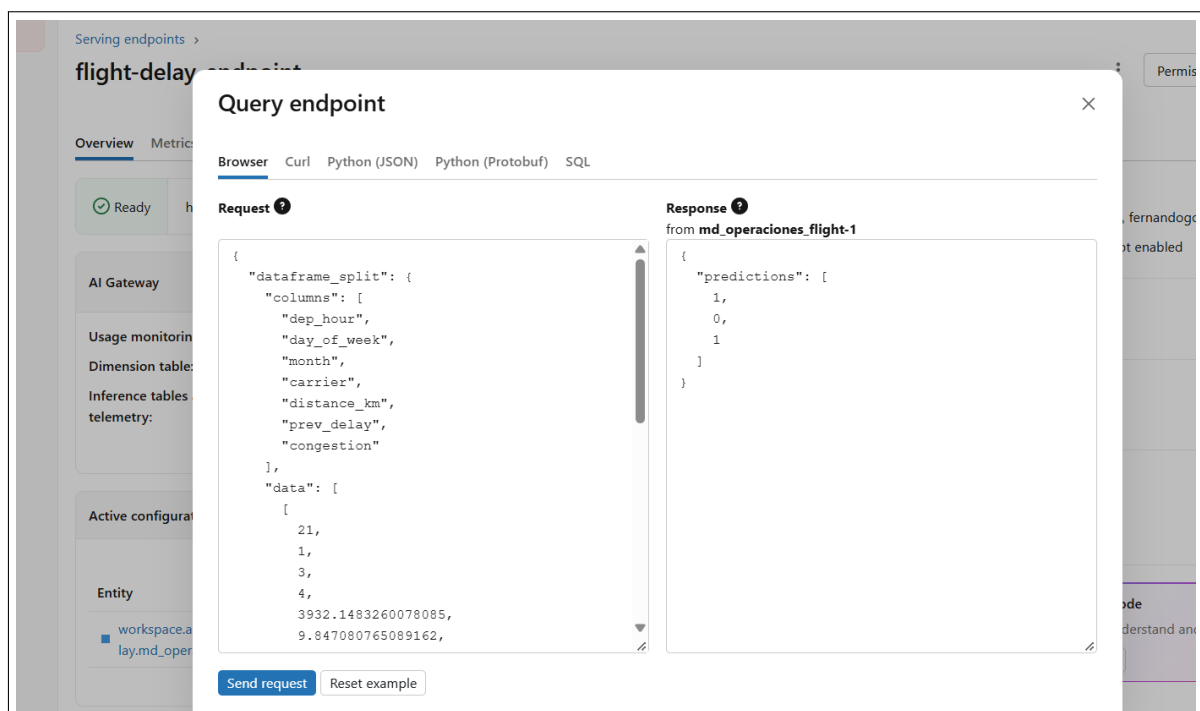
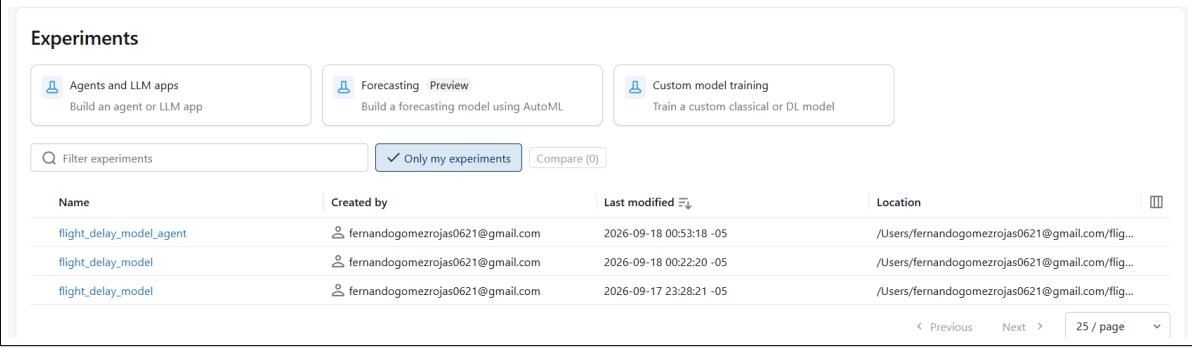


Figure 23: Consulta al servicio. La entrada corresponde a las variables declaradas en el contrato de características. En operación sostenida, su resolución en el momento de la solicitud requiere un almacén de características en línea; sin esa pieza, la responsabilidad de calcularlas se traslada al consumidor.

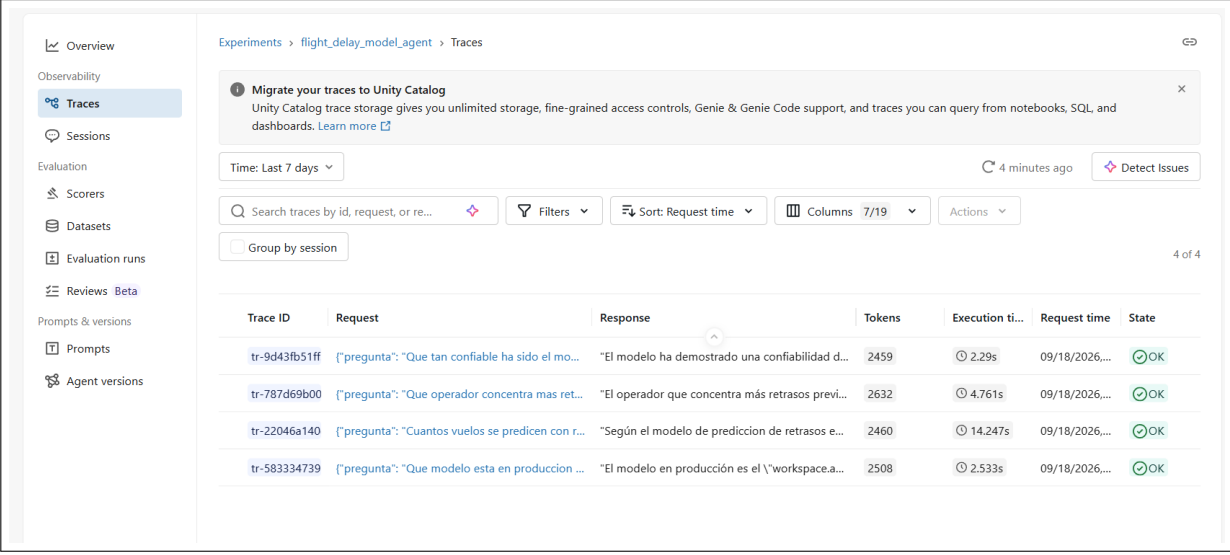
A.8 Consumo asistido por agentes



The screenshot shows the 'Experiments' section of a dashboard. It includes three cards for 'Agents and LLM apps', 'Forecasting', and 'Custom model training'. Below these is a search bar and a table of experiments.

Name	Created by	Last modified	Location
flight_delay_model_agent	fernandogomezrojas0621@gmail.com	2026-09-18 00:53:18 -05	/Users/fernandogomezrojas0621@gmail.com/flig...
flight_delay_model	fernandogomezrojas0621@gmail.com	2026-09-18 00:22:20 -05	/Users/fernandogomezrojas0621@gmail.com/flig...
flight_delay_model	fernandogomezrojas0621@gmail.com	2026-09-17 23:28:21 -05	/Users/fernandogomezrojas0621@gmail.com/flig...

Figure 24: El agente registra su propio experimento, separado del ciclo de entrenamiento. La capa de consumo se observa con el mismo instrumental que el modelo, pero como componente independiente.



The screenshot shows the 'Traces' section of a dashboard. It includes a sidebar with navigation options like 'Overview', 'Traces', 'Sessions', 'Evaluation', 'Scorers', 'Datasets', 'Evaluation runs', 'Reviews', 'Prompts', and 'Agent versions'. The main area displays a table of traces with columns for Trace ID, Request, Response, Tokens, Execution time, Request time, and State.

Trace ID	Request	Response	Tokens	Execution ti...	Request time	State
tr-9d43fb51ff	["pregunta": "Que tan confiable ha sido el mo...	"El modelo ha demostrado una confiabilidad d...	2459	2.29s	09/18/2026...	OK
tr-787d69b00	["pregunta": "Que operador concentra mas ret...	"El operador que concentra más retrasos previ...	2632	4.761s	09/18/2026...	OK
tr-22046a140	["pregunta": "Cuantos vuelos se predicen con r...	"Según el modelo de prediccion de retrasos e...	2460	14.247s	09/18/2026...	OK
tr-583334739	["pregunta": "Que modelo esta en produccion ...	"El modelo en producción es el \"workspace.a...	2508	2.533s	09/18/2026...	OK

Figure 25: Registro de interacciones. Cada consulta queda trazada con su latencia, consumo y estado. El registro permite auditar la capa que traduce predicciones a lenguaje natural, y detectar degradaciones de latencia o costo antes de que las reporte el usuario.

El detalle de una traza individual, con la pregunta, la herramienta invocada y la respuesta, se muestra en la figura 5.

Figure 26: Consulta agregada sobre la tabla de inferencia. La respuesta cita el ambiente del que proceden las cifras, lo que evita que un dato de desarrollo se interprete como productivo.

Figure 27: Consulta por segmento de negocio. El agente distingue entre volumen absoluto y tasa, que son preguntas distintas con respuestas distintas, y reporta ambas cuando la pregunta es ambigua.

Figure 28: Consulta sobre confiabilidad. La respuesta se construye comparando predicción contra resultado real registrado en la tabla, no sobre las métricas de entrenamiento. La distinción es relevante: el desempeño observado en producción es el que importa para la decisión operativa.

A.9 Respuestas y secuencia de razonamiento

Las figuras anteriores muestran el registro de la interacción. Esta sección documenta el contenido: qué preguntó el negocio, qué herramienta resolvió la consulta y qué devolvió el agente.

Secuencia observada. Cada traza presenta la misma estructura de tres tramos: una primera llamada al modelo de lenguaje que interpreta la pregunta y decide qué herramienta invocar; la ejecución de esa herramienta contra el catálogo; y una segunda llamada que redacta la respuesta a partir del resultado obtenido. El modelo no accede a los datos: accede a herramientas.

Tramo	Componente	Función
Interpretación	Modelo de lenguaje	Traduce la pregunta de negocio a una invocación de herramienta con sus argumentos
Resolución	Herramienta declarada	Ejecuta una consulta acotada sobre el catálogo y devuelve datos estructurados
Redacción	Modelo de lenguaje	Construye la respuesta exclusivamente a partir del resultado devuelto

Table 10: Estructura de una interacción. La separación entre interpretación y resolución es lo que impide que el modelo genere cifras.

Transcripción de las consultas.

Pregunta	Herramienta	Respuesta
¿Qué modelo está en producción y pasó la validación?	<code>modelo_en_produccion</code>	Identifica el modelo del catálogo productivo en su versión vigente, cita el identificador de ejecución que lo vincula con su entrenamiento, informa veredicto favorable y nombra la métrica principal empleada.
¿Cuántos vuelos se predicen con retraso y cuál es la tasa?	<code>resumen_predicciones</code>	Reporta 294 retrasos previstos sobre 500 operaciones evaluadas, equivalente a una tasa del 58,8%, citando explícitamente el ambiente del que proceden las cifras.
¿Qué operador concentra más retrasos previstos?	<code>retrasos_por_segmento</code>	Distingue entre volumen absoluto y tasa: identifica al operador con mayor número de retrasos previstos y señala que otro operador distinto presenta la tasa más alta.
¿Qué tan confiable ha sido el modelo y cuántos falsos positivos genera?	<code>desempeno_observado</code>	Reporta 88% de aciertos y 31 falsos positivos sobre 500 casos, calculados comparando predicción contra resultado real registrado, no sobre métricas de entrenamiento.

Table 11: Consultas resueltas por el agente. Cada cifra procede de una consulta al catálogo registrada en su traza.

Observaciones sobre el comportamiento.

- **Selección correcta de herramienta en las cuatro consultas.** No hubo invocaciones innecesarias ni encadenamientos redundantes. Con un catálogo pequeño de herramientas bien descritas la selección es fiable; el riesgo crece con herramientas de alcance solapado.
- **Desambiguación explícita.** Ante una pregunta que admitía dos lecturas —mayor volumen frente a mayor tasa— el agente reportó ambas en lugar de elegir una. Es el comportamiento deseable cuando la diferencia tiene consecuencia operativa.
- **Citación del ambiente.** Las respuestas indican de qué ambiente proceden las cifras. Sin esa precaución, un dato de desarrollo puede interpretarse como productivo, riesgo real cuando el mismo agente se despliega en los tres ambientes.

- **Distinción entre desempeño declarado y observado.** Ante la pregunta por confiabilidad, el agente usó la comparación contra resultado real y no la métrica registrada en entrenamiento. El desempeño en producción es el relevante para la decisión operativa.

Nota sobre el modelo de lenguaje empleado. El agente descubre en tiempo de ejecución qué puntos de acceso hay disponibles en el espacio de trabajo y selecciona uno según un orden de preferencia declarado. Esto evita acoplar el código a un modelo concreto, que cambia con frecuencia. La consecuencia para la evidencia es que el modelo efectivamente utilizado queda registrado en el experimento del agente y en cada traza, no en el código.

A.10 Observaciones de la implementación

- **El contrato detiene la ejecución.** El ciclo no arrancó mientras el catálogo de características estuvo vacío. La alternativa —ejecutar con una definición por defecto— habría producido un artefacto sin sentido que habría llegado a validación. El fallo temprano es comportamiento deseado.
- **La parametrización se verifica ejecutando, no leyendo.** Un valor de esquema incrustado en una librería compartida impidió el despliegue en un ambiente con topología distinta. El resto del proyecto estaba correctamente parametrizado; la fuga solo se manifestó al cambiar de destino. De ahí la conveniencia de incluir un despliegue en ambiente alterno como parte de la integración continua.
- **Las dependencias del ciclo no son errores.** El monitor requiere la tabla de inferencia; la tabla requiere un modelo con alias; el alias requiere una validación aprobada. Ese encadenamiento es el gobierno funcionando, y determina el orden correcto de las operaciones en un ambiente nuevo.
- **Los casos borde deben tener salida explícita.** Una ejecución de inferencia sobre un rango sin registros terminó con un estado declarado en lugar de producir una tabla vacía. Sin ese control, la ausencia de predicciones se habría descubierto por el reporte del negocio.
- **La observabilidad requiere historia.** Instrumentar el monitoreo no equivale a tener señal. Las métricas de deriva necesitan al menos dos ventanas comparables, y la estabilidad de la métrica depende del volumen por ventana.